Selenium with Python: compact cheatsheet

STEP 1 Meet Selenium

1.1 What Selenium is and where it came from

What Selenium is An umbrella project of tools and libraries that automate real browsers the way a user would. At its core, a browser user agent library.

Three parts WebDriver (library plus browser driver, for code), IDE (record and play back extension), Grid (many sessions on many machines).

Not a test framework Selenium drives the browser. Assertions, test structure and reports come from JUnit, pytest, Mocha and similar tools.

Key dates Selenium 2.0 on 8 July 2011, 3.0 on 13 October 2016, W3C Recommendation on 5 June 2018, Selenium 4.0 on 13 October 2021.

Program shape Start a session, send the browser somewhere, ask or act, and always quit at the end.

```
driver = webdriver.Chrome()
```
Start Chrome. Selenium Manager finds a matching chromedriver, then a new session begins.

```
driver.get("https://www.selenium.dev/selenium/web/web-form.html")
```
Open the page and wait until it has loaded.

TRAP Selenium only controls the browser. It does not decide whether a test passed, it does not write reports and it does not give you Given, When, Then syntax. Those come from a test framework such as JUnit, pytest or Mocha. Chapter 5.1 joins the two.

1.2 Architecture and the W3C WebDriver protocol

Three programs Your code and the binding (local end), the driver (remote end), the browser. The driver is written by the browser vendor.

HTTP and JSON Every command is one HTTP request with a fixed method and path. Answers are {"value": ...} on success or {"value": {"error", "message", "stacktrace"}} on failure.

Session id POST /session returns a session id. Every later path contains it. DELETE /session/{id} ends it.

Element reference Elements travel as {"element-6066-11e4-a52e-4f735466cecf": "id"}. Bindings turn id, name and class name into CSS selectors.

BiDi Classic WebDriver is request and response. WebDriver BiDi adds a WebSocket so the browser can send events. CDP access is temporary until BiDi covers it.

```
time.sleep(1)
```
Give it a moment to open port 9515.

```
def call(method, path, body=None):
```
Send one W3C command and return the part of the answer under "value".

```
data = json.dumps(body).encode() if body is not None else None
```
POST bodies are JSON objects. GET and DELETE have none.

```
reply = json.load(urllib.request.urlopen(req))
```
Send it and read the JSON answer.

```
return reply["value"]
```
Every success answer keeps its result under "value".

```
KEY = "element-6066-11e4-a52e-4f735466cecf"
```
The fixed key the W3C spec uses for element references.

```
call("POST", s + "/timeouts", {"implicit": 2000})
```
Set Timeouts: Find Element may keep looking for 2 seconds, so the next page has time to load.

```
call("POST", s + "/url", {"url": "https://www.selenium.dev/selenium/web/web-form.html"})
```
Navigate To.

```
call("GET", s + "/title")
```
Get Title.

```
box = call("POST", s + "/element", {"using": "css selector", "value": "[name='my-text']"})[KEY]
```
Find Element.

```
call("POST", s + "/element/" + box + "/value", {"text": "Selenium"})
```
Element Send Keys.

```
call("POST", s + "/element/" + button + "/click", {})
```
Element Click submits the form.

```
call("GET", s + "/element/" + message + "/text")
```
Get Element Text on the next page.

```
call("DELETE", s)
```
Delete Session closes Chrome.

```
driver.terminate()
```
Stop chromedriver.

```
logging.basicConfig(level=logging.DEBUG, format="%(message)s", stream=sys.stdout)
```
Print DEBUG messages, which include every HTTP request the binding sends.

```
for name in ("urllib3", "selenium.webdriver.common"):
```
Hide two noisy loggers so only the protocol messages remain.

```
box = driver.find_element(By.NAME, "my-text")
```
POST /session/{id}/element. Watch how By.NAME becomes a CSS selector.

```
box.send_keys("Hi")
```
POST /session/{id}/element/{element id}/value

Success HTTP 200 and {"value": result}

Error HTTP 4xx or 5xx and {"value": {"error", "message", "stacktrace"}}

1.3 Setup, Selenium Manager and your first script

Install Python: pip install selenium (3.10+).

Selenium Manager Ships with Selenium since 4.6. A fallback that discovers the browser version, resolves, downloads and caches the driver in ~/.cache/selenium, and since 4.11 can download browsers.

Driver mismatch An old driver on PATH or given by hand gives session not created. Delete it and let Selenium Manager match the driver.

Eight components Start session, act on browser, ask the browser, wait strategy, find element, act on element, ask the element, end session.

Always quit Put quit in finally, or use a Python with block. quit closes every window and stops the driver, close only closes one window.

```
service = Service(executable_path="/opt/drivers/chromedriver-147")
```
Point Selenium at an old chromedriver 147 on purpose. Selenium Manager is now skipped.

```
driver = webdriver.Chrome(service=service)
```
Chrome on this machine is version 153, so New Session fails.

```
except SessionNotCreatedException as e:
```
The driver answered with the W3C error code session not created.

```
title = driver.title
```
3. Request browser information: the page title.

```
driver.implicitly_wait(0.5)
```
4. Establish a waiting strategy. Python takes seconds, so 0.5 means half a second.

```
text_box = driver.find_element(by=By.NAME, value="my-text")
```
5. Find an element by its name attribute.

```
submit_button = driver.find_element(by=By.CSS_SELECTOR, value="button")
```
5. Find the first button with a CSS selector.

```
text_box.send_keys("Selenium")
```
6. Take action on an element: type into the box.

```
submit_button.click()
```
6. Click Submit. The browser loads the next page.

```
message = driver.find_element(by=By.ID, value="message")
```
Find the message on the new page.

```
value = message.text
```
7. Request element information: its visible text.

```
with webdriver.Chrome() as driver:
```
A with block calls driver.quit() for you when the block ends, even after an error.

```
driver.find_element(By.ID, "no-such-id")
```
This element does not exist, so this line raises.

```
except NoSuchElementException as e:
```
Catch the error so we can report it.

TRAP If you pass a driver path yourself, Selenium trusts you and never calls Selenium Manager. The documentation says that with Selenium 4.6 or newer you should not need to set a driver location at all. Delete old drivers from PATH and let Selenium Manager do the job.

TRAP The documentation itself warns that an implicit wait is rarely the best solution. It is used here because it is the easiest to show. Chapter 4.1 replaces it with explicit waits.

STEP 2 Control the browser

2.1 Sessions, options and timeouts

Options Since Selenium 4, describe sessions with Options classes. They become W3C capabilities, and browser specific ones carry a prefix such as goog:chromeOptions.

Page load strategy normal waits for complete (load), eager for interactive (DOMContentLoaded), none for the first download. It covers navigation by address, not clicks.

Default timeouts Implicit 0, page load 300,000 ms, script 30,000 ms.

Service Starts and stops a local driver: file, port, arguments and logs. Not used for remote sessions.

Granted capabilities Read them after New Session to learn the real browser version, platform and defaults such as dismiss and notify.

```
options = webdriver.ChromeOptions()
```
An Options object describes the session you want. browserName is already chrome.

```
options.add_argument("--headless=new")
```
A Chrome command line switch: run without a visible window.

```
options.add_argument("--window-size=1280,800")
```
Another switch: the size of the browser window.

```
options.page_load_strategy = "eager"
```
A W3C capability: get() returns at DOMContentLoaded.

```
driver = webdriver.Chrome(options=options)
```
New Session is sent with these capabilities.

```
caps = driver.capabilities
```
A plain dict of what the driver actually granted.

```
for key in ("browserName", "browserVersion", "platformName", "pageLoadStrategy", "unhandledPromptBehavior"):
```
Print a few of the standard keys.

```
t = driver.timeouts
```
The timeouts of this session. Python reports them in seconds.

```
driver.set_script_timeout(1)
```
Allow scripts only one second from now on.

```
driver.execute_async_script("/* never calls its callback */")
```
An async script must call its callback. This one never does.

```
except TimeoutException as e:
```
Python raises TimeoutException for the W3C error script timeout.

```
service = Service(port=9600)
```
A Service describes how to start chromedriver: here, on port 9600.

```
with urllib.request.urlopen("http://localhost:9600/status") as reply:
```
GET /status is a W3C command that needs no session.

Page load 300,000 ms

TRAP Switching to eager or none to make tests faster is a trade. Every element that arrives later now needs an explicit wait, or the test becomes flaky. Chapter 4.1 shows how to write those waits.

EXAM These timeouts belong to the browser session. Explicit waits such as WebDriverWait are different: they run in your code and poll the browser. Mixing an implicit wait with explicit waits can give surprising wait times, which chapter 4.1 explains.

2.2 Navigation, window size, screenshots and PDF

Navigation get or navigate().to, back, forward, refresh. Python spells them driver.back(), driver.forward(), driver.refresh().

Where am I Title and current URL. Check them when a test depends on a redirect or on history, because back on the first page raises no error.

Window rect Set and get size and position. An exact size is more portable than maximize, which needs a window manager.

Screenshots Take Screenshot (viewport) and Take Element Screenshot return base64 PNG. Save one when a test fails, before quit.

Print Page Returns a base64 PDF. Use PrintOptions for orientation, ranges, size, margins and scale. Chromium needs headless.

```
driver.get("https://www.selenium.dev/selenium/web/xhtmlTest.html")
```
A second page. History now holds two entries.

```
driver.back()
```
Press the Back button.

```
driver.forward()
```
Press the Forward button.

```
driver.refresh()
```
Reload the current page.

```
driver.set_window_size(1024, 768)
```
Restore the window and make it 1024 by 768 pixels.

```
driver.set_window_position(40, 30)
```
Move its top left corner to x 40, y 30 on the screen.

```
size = driver.get_window_size()
```
A dict with width and height.

```
where = driver.get_window_position()
```
A dict with x and y.

```
def png_size(path):
```
A PNG keeps its width and height at bytes 16 to 24 of the file.

```
driver.save_screenshot("page.png")
```
Take Screenshot of the viewport and save it as a PNG.

```
button.screenshot("button.png")
```
Take Element Screenshot: just the area of the button.

```
print_options = PrintOptions()
```
Paper settings: orientation, page ranges, size, margins, scale, background.

```
pdf = base64.b64decode(driver.print_page(print_options))
```
The W3C Print Page command answers with base64, so decode it.

TRAP After refresh or any navigation, the elements you found before belong to the old page. Using them raises stale element reference. Find them again on the new page. Chapter 5.4 covers this error in depth.

TRAP Maximize and minimize need a desktop window manager. On a bare Linux build server they can fail with an unknown error, which is what happened when this page was built. Setting an exact size with Set Window Rect or the --window-size switch works everywhere and gives the same layout on every machine.

STEP 3 Find and use elements

3.1 Locators: how Selenium finds an element

Eight strategies id, name, class name, css selector, xpath, tag name, link text, partial link text. find element returns the first match.

Five on the wire W3C defines css selector, link text, partial link text, tag name and xpath. Bindings turn id, name and class name into CSS.

CSS and XPath CSS: #id, .class, tag[attr="v"], parent > child. XPath adds text matching and moving up to a parent. Prefer relative XPath (//).

Relative locators above, below, toLeftOf, toRightOf and near (50 px), built on getBoundingClientRect. They depend on layout, so check what they return.

Preference Unique stable id, then a short CSS selector, then XPath. Avoid absolute paths, styling classes and generated ids. Consider data-test attributes.

```
def show(how, e):
```
A helper that prints which element a locator found.

```
show("CLASS_NAME", driver.find_element(By.CLASS_NAME, "information"))
```
The first element whose class list contains information.

```
show("CSS_SELECTOR", driver.find_element(By.CSS_SELECTOR, "#fname"))
```
CSS: # means id, so the input with id fname.

```
show("ID", driver.find_element(By.ID, "lname"))
```
The element whose id attribute is lname.

```
show("NAME", driver.find_element(By.NAME, "newsletter"))
```
The element whose name attribute is newsletter.

```
show("LINK_TEXT", driver.find_element(By.LINK_TEXT, "Selenium Official Page"))
```
A link whose whole visible text matches.

```
show("PARTIAL_LINK_TEXT", driver.find_element(By.PARTIAL_LINK_TEXT, "Official"))
```
A link whose visible text contains Official.

```
show("TAG_NAME", driver.find_element(By.TAG_NAME, "a"))
```
The first a element on the page.

```
show("XPATH", driver.find_element(By.XPATH, "//input[@value='f']"))
```
XPath: any input whose value attribute is f.

```
show("above lname", driver.find_element(locate_with(By.TAG_NAME, "input").above({By.ID: "lname"})))
```
An input above the lname box.

TRAP Each relative locator sends the page a script to run, so it is slower than a plain CSS selector. Prefer an id or a short CSS selector, and keep relative locators for elements with no good attributes.

EXAM Keep locators in one place, apart from the steps that use them. The documentation recommends declaring them separately, which is exactly what a page object does. Chapter 5.2 shows the pattern.

3.2 Finding elements: one, many, scoped and in shadow DOM

One or many find element returns the first match or raises no such element. find elements returns all matches or an empty list.

Search context You can search from the driver, from an element or from a shadow root. CSS from an element stays inside it.

XPath trap An XPath starting with // always starts at the document, even when called on an element. Use .// to search below the element.

Shadow DOM Find the host, get its shadow root, then search with a CSS selector. Chrome refuses XPath there.

Active element switchTo().activeElement() returns the focused element. Use it to test keyboard flow such as Tab order.

```
inputs = driver.find_elements(By.TAG_NAME, "input")
```
Plural: a list of every input, in page order.

```
for e in inputs:
```
Walk the list like any Python list.

```
none = driver.find_elements(By.ID, "email")
```
No match: an empty list, never an error.

```
driver.find_element(By.ID, "email")
```
Singular with no match: an exception.

```
form = driver.find_element(By.TAG_NAME, "form")
```
First find the parent element.

```
last = form.find_element(By.ID, "lname")
```
Two commands: find the form, then find inside it.

```
same = driver.find_element(By.CSS_SELECTOR, "form #lname")
```
The optimised version: one CSS selector, one command.

```
host = driver.find_element(By.CSS_SELECTOR, "custom-checkbox-element")
```
The shadow host: the element the hidden tree is attached to.

```
shadow_root = host.shadow_root
```
Get Element Shadow Root. A ShadowRoot object with find_element and find_elements.

```
checkbox = shadow_root.find_element(By.CSS_SELECTOR, box)
```
Search inside the shadow tree.

```
shadow_root.find_element(By.XPATH, ".//input")
```
XPath inside a shadow root.

```
driver.find_element(By.NAME, "my-text").click()
```
Clicking a text box gives it the focus.

```
driver.switch_to.active_element.send_keys(Keys.TAB)
```
Press Tab on whatever has the focus, as a user would.

TRAP find elements never waits for the first element to appear unless an implicit wait is set, and then it returns as soon as at least one matches. An empty list can mean the page is not ready yet. Chapter 4.1 shows how to wait properly.

3.3 Interacting with elements and reading them

Five commands click, send keys, clear, submit and select. Prefer clicking the submit button over submit(). send keys appends, so clear first.

Built in checks Before acting the driver scrolls the element into view and checks it is interactable. Errors: element not interactable, invalid element state, element click intercepted.

Attribute vs property DOM attribute is the HTML, DOM property is the live value. The classic getAttribute mixes both.

Select and upload Select works only on select elements: by visible text, value or index. Upload by sending the full path to input type="file".

```
driver.implicitly_wait(2)
```
As in chapter 1.3: let finds retry for up to 2 seconds, so the page after Submit has time to load.

```
text.send_keys("first try")
```
Element Send Keys: types into the box, adding to what is already there.

```
text.clear()
```
Element Clear: empties the box.

```
text.send_keys("Selenium")
```
Type the real value.

```
check2.click()
```
Element Click: ticks it, as a mouse click would.

```
driver.find_element(By.ID, "my-radio-2").click()
```
Clicking the second radio button unselects the first one.

```
driver.find_element(By.CSS_SELECTOR, "button").click()
```
Click Submit. The documentation prefers this to the submit() method.

```
def attempt(what, step):
```
Run one step and report the error it raises, if any.

```
except WebDriverException as e:
```
Every Selenium error is a WebDriverException.

```
attempt("type into disabled", lambda: driver.find_element(By.NAME, "my-disabled").send_keys("x"))
```
A disabled input cannot take keyboard input.

```
attempt("clear readonly", lambda: driver.find_element(By.NAME, "my-readonly").clear())
```
A readonly input cannot be reset.

```
driver.execute_script(
```
Add a cookie banner that covers the bottom of the page, as many real sites do.

```
attempt("click under banner", lambda: driver.find_element(By.CSS_SELECTOR, "button").click())
```
The banner sits over the centre of the button.

```
text.send_keys("hello")
```
Type, so the live value differs from the HTML.

```
single = Select(driver.find_element(By.NAME, "selectomatic"))
```
Wrap a select element in a Select object.

```
single.select_by_visible_text("Four")
```
Pick by the text the user sees.

```
single.select_by_value("two")
```
Pick by the value attribute.

```
single.select_by_index(0)
```
Pick by position, counting from 0.

```
multi = Select(driver.find_element(By.ID, "multi"))
```
A select with the multiple attribute.

```
multi.deselect_by_value("eggs")
```
Only multiple lists can deselect.

```
file.write_text("quarterly numbers")
```
Make a small file to upload.

```
upload = driver.find_element(By.NAME, "my-file")
```
The input with type="file".

```
upload.send_keys(str(file.resolve()))
```
Send the full path. No dialog opens, the file is simply attached.

STEP 4 Synchronise and control

4.1 Waiting strategies: implicit, explicit and fluent

Implicit wait A session setting, default 0. Every find keeps looking up to that long and returns as soon as the element appears. It only waits for presence.

Explicit wait A loop with a condition, timeout and polling interval.

Expected conditions presence, visibility, clickable, text present, title contains, staleness. Wait for visibility before typing.

Never mix Implicit plus explicit makes waits unpredictable: 10 s with 15 s can time out after about 20 s. Keep implicit at 0 and use explicit waits.

```
driver.find_element(By.ID, "adder").click()
```
The page adds a red box, but only 1 second after the click.

```
driver.find_element(By.ID, "box0")
```
Our code asks at once. No wait is set, so the driver checks only once.

```
box = driver.find_element(By.ID, "box0")
```
Not there yet, so the driver keeps checking until the box appears.

```
revealed = driver.find_element(By.ID, "revealed")
```
The input is in the page already, just hidden.

```
driver.find_element(By.ID, "reveal").click()
```
The page shows it 1 second later.

```
wait = WebDriverWait(driver, timeout=2)
```
Wait at most 2 seconds. Python checks every 0.5 s.

```
wait.until(lambda d: revealed.is_displayed())
```
A lambda: keep calling it until it returns something true.

```
revealed.send_keys("Displayed")
```
Safe now: the input is displayed, so it can take keys.

```
box = wait.until(EC.presence_of_element_located((By.ID, "box0")))
```
A ready made condition. Note the locator is one tuple. It returns the element.

```
field = wait.until(EC.visibility_of_element_located((By.ID, "revealed")))
```
Present is not enough for typing. Wait until it is visible.

```
WebDriverWait(driver, 1).until(EC.presence_of_element_located((By.ID, "box1")), "the second box never came")
```
Nobody clicked again, so box1 never appears. The second argument is our message.

```
errors = [ElementNotInteractableException]
```
The errors that mean not yet, try again.

```
wait = WebDriverWait(driver, timeout=2, poll_frequency=0.3, ignored_exceptions=errors)
```
Give up after 2 s, check every 0.3 s.

```
wait.until(lambda d: revealed.send_keys("Displayed") or True)
```
Do the action inside the condition. send_keys returns None, so "or True" reports success.

```
WebDriverWait(driver, 3).until(EC.presence_of_element_located((By.ID, "box9")))
```
Explicit wait of 3 seconds for an element that never comes.

4.2 Windows, tabs, frames and alerts

Windows Each window or tab has a handle. Save the original, switch to the new one, and switch back after close(). newWindow("tab") opens and switches in one step.

Frames Switch into an iframe by element, name or id, or index before finding inside it. defaultContent goes to the top, parentFrame up one level.

Title in frames Get Title always reports the top level page, even from inside a frame.

Alerts alert, confirm and prompt: getText, accept, dismiss, sendKeys. Wait with alert is present when it appears late.

Forgotten pop ups unhandledPromptBehavior defaults to dismiss and notify: the pop up is closed and the next command raises unexpected alert open.

```
original = driver.current_window_handle
```
The id of the current window. It stays the same for the whole session.

```
driver.find_element(By.NAME, "windowTwo").click()
```
This link has target="_blank", so it opens a new window.

```
WebDriverWait(driver, 2).until(EC.number_of_windows_to_be(2))
```
Wait until the second window exists.

```
for handle in driver.window_handles:
```
Every open window, in the order they were opened.

```
driver.switch_to.window(handle)
```
Switch to the one that is not the original.

```
driver.close()
```
Close only this window.

```
driver.switch_to.window(original)
```
Switch back, or the next command fails with no such window.

```
driver.switch_to.new_window("tab")
```
Selenium 4: open a new tab and switch to it in one step.

```
driver.switch_to.frame(frame)
```
1. Switch with a WebElement: the most flexible way.

```
driver.switch_to.default_content()
```
Leave the frame: back to the top document.

```
driver.switch_to.frame("iframe1-name")
```
2. Switch with the frame name or id.

```
driver.switch_to.frame(0)
```
3. Switch with an index: the first frame of the page.

```
driver.find_element(By.ID, "alert").click()
```
The page calls alert('cheese').

```
alert = wait.until(EC.alert_is_present())
```
Wait for the alert, then get a handle on it.

```
alert.accept()
```
Press OK.

```
driver.find_element(By.ID, "confirm").click()
```
A confirm box: OK or Cancel.

```
confirm = driver.switch_to.alert
```
Switch to it without waiting, because it opens at once.

```
confirm.dismiss()
```
Press Cancel.

```
driver.find_element(By.ID, "prompt").click()
```
A prompt asks for text.

```
prompt.send_keys("Selenium")
```
Type the answer into the prompt.

```
driver.find_element(By.ID, "slow-alert").click()
```
This one opens 200 ms later.

```
driver.title
```
Any command while an alert is open.

TRAP After close() you are still pointing at the closed window. The documentation warns that the next command raises no such window. Always switch back to a handle you saved before.

4.3 The Actions API: mouse, keyboard and wheel

Build then perform An action chain is sent in one request when perform() is called. Nothing happens before that.

Ticks Each input source has one action per tick. A tick lasts as long as its slowest action.

Mouse doubleClick, contextClick, moveToElement, clickAndHold, dragAndDrop, moveByOffset. The driver remembers the pointer position between chains.

Keyboard keyDown and keyUp for modifiers, sendKeys for text. Use Command on a Mac and Control elsewhere. Release every key you press.

Wheel scrollToElement, scrollByAmount, scrollFromOrigin. Wait for the scroll result, because it can land after perform() returns.

```
ActionChains(driver).double_click(box).perform()
```
Build a chain, then perform() sends it to the browser in one request.

```
ActionChains(driver).context_click(box).perform()
```
A right click.

```
ActionChains(driver).move_to_element(driver.find_element(By.ID, "hover")).perform()
```
Move to the centre of the element. Nothing is clicked.

```
ActionChains(driver).drag_and_drop(source, target).perform()
```
Move to source, press, move to target, release.

```
builder = ActionBuilder(driver)
```
The lower level builder. Python has no move_to_location on ActionChains.

```
builder.pointer_action.move_to_location(8, 12)
```
A point measured from the top left of the viewport.

```
ActionChains(driver).move_by_offset(13, 15).perform()
```
From wherever the pointer is now. The driver remembered it between chains.

```
.perform()
```
Shift is held for abc only. send_keys goes to the box that has focus.

```
cmd_ctrl = Keys.COMMAND if sys.platform == "darwin" else Keys.CONTROL
```
Select all is Command A on a Mac and Control A elsewhere.

```
ActionChains(driver).send_keys_to_element(box, "Selenium!").perform()
```
Clicks the element first, then types.

```
frame = driver.find_element(By.TAG_NAME, "iframe")
```
The page starts with a 5000 px tall block, so the iframe is far below.

```
ActionChains(driver).scroll_to_element(frame).perform()
```
Scroll until the bottom of the element is in view.

```
ActionChains(driver).scroll_by_amount(0, 300).perform()
```
Like turning the wheel: 300 px down from the top left of the viewport.

```
WebDriverWait(driver, 2).until(lambda d: d.execute_script("return window.scrollY") >= 300)
```
The scroll can land after perform returns, so wait for it.

```
ActionChains(driver).scroll_from_origin(ScrollOrigin.from_element(inner), 0, 200).perform()
```
Turn the wheel with the pointer over the frame, so the frame scrolls.

TRAP Actions type into the focused element. If nothing has focus, the keys go to the page body and the test seems to type into nothing. Element sendKeys focuses the element itself, which is why it is the better choice for plain typing.

4.4 JavaScript execution and cookies

executeScript The script runs as a function body. Use return to get a value and arguments[n] to read what you pass in. Elements go in and come back as WebElements.

Return types Whole numbers are Long in Java and int in Python, objects are Map and dict, undefined is null or None.

Async scripts executeAsyncScript adds a callback as the last argument. A returned promise is also awaited. The script timeout, 30 s by default, limits both.

Cookies Visit the domain first, then add. httpOnly hides a cookie from page scripts. deleteAllCookies gives a clean start.

```
driver.execute_script("arguments[0].click();", driver.find_element(By.ID, "updatediv"))
```
A click done by JavaScript. It skips the checks a real click makes.

```
driver.set_script_timeout(0.2)
```
How long a script may run, in seconds. The default is 30.

```
driver.add_cookie({"name": "key", "value": "value"})
```
The browser is still on an empty start page with no domain.

```
driver.get("https://www.selenium.dev/selenium/web/blank.html")
```
Visit the domain first. A small page is enough.

```
c = driver.get_cookie("key")
```
Read one cookie by name. It comes back as a dict.

```
driver.add_cookie({"name": "foo", "value": "bar", "sameSite": "Strict"})
```
Any other field goes in the same dict.

```
driver.add_cookie({"name": "secret", "value": "s", "httpOnly": True})
```
httpOnly hides the cookie from page JavaScript.

```
driver.delete_all_cookies()
```
A clean start, for example before logging in as another user.

STEP 5 Build a test suite

5.1 Test frameworks: JUnit 5, pytest and Mocha

Framework jobs Selenium drives the browser. JUnit 5, pytest or Mocha find the tests, run setup and teardown, assert and report.

Fresh browser Start the driver in the before each step and quit it in the after each step, so tests stay independent and can run in any order.

Teardown always runs @AfterEach, code after yield and afterEach run also when the test failed, so quit belongs there.

Data-driven @ParameterizedTest with @CsvSource, @pytest.mark.parametrize, or a loop around it() in Mocha. One result per row.

Screenshot on failure Take it after the test failed and before quit: a pytest makereport hook plus fixture.

```
@pytest.fixture
```
A fixture is pytest's setup and teardown in one function.

```
yield driver
```
Everything before yield is setup. The test runs here.

```
def test_page_has_the_right_title(driver):
```
Asking for a parameter called driver makes pytest run the fixture.

```
assert driver.title == "Web form"
```
A plain assert. pytest shows both sides when it fails.

```
@pytest.mark.parametrize("user, password, expected", [
```
Each tuple is one run of the test.

```
def test_login(driver, user, password, expected):
```
pytest passes the columns in as parameters, next to the fixture.

```
alert = WebDriverWait(driver, 2).until(EC.alert_is_present())
```
The page answers with an alert.

```
@pytest.hookimpl(wrapper=True, tryfirst=True)
```
A hook that sees every test report as pytest builds it.

```
setattr(item, "rep_" + report.when, report)
```
Keep the report on the test item: rep_setup, rep_call and rep_teardown.

```
if request.node.rep_call.failed:
```
Did the test body fail?

```
name = f"failure-{request.node.name}.png"
```
Name the file after the test.

```
driver.save_screenshot(name)
```
Take the picture before quit.

```
def test_title_is_wrong_on_purpose(driver):
```
The driver fixture comes from conftest.py.

```
assert driver.title == "Web form!"
```
Wrong on purpose, so the test fails.

5.2 Page objects and page components

Page object A class that is the interface to one page. Tests call its methods, so a change to the page is fixed in one place.

No assertions inside Assertions belong to the test. The only check in a page object is that the right page is loaded.

Return the next page A method that moves to another page returns that page object. A method that stays may return itself.

Components A page component object models a repeated part, such as a field or a product card, and searches only inside its root element.

Waits inside Put waits in the page object, for example a constructor that waits for its title after a navigation.

```
FIELDS = (By.CSS_SELECTOR, "label.form-label")
```
Locators live here and nowhere else.

```
if driver.title != "Web form":
```
The one check a page object should make: am I on the right page?

```
return [Field(e) for e in self.driver.find_elements(*self.FIELDS)]
```
Wrap every label element in a Field component.

```
class Field:
```
A page component: one label and the control inside it.

```
return self.root.text.split("\n")[0].strip()
```
The first line of the label is its name.

```
return self.root.find_element(By.CSS_SELECTOR, "input, textarea, select")
```
Search inside this component only.

```
return self
```
Returning itself allows field(...).fill(...).fill(...).

```
WebDriverWait(driver, 3).until(EC.title_is("Web form - target page"))
```
The page arrives after a navigation, so wait for it instead of checking once.

```
form = WebFormPage.open(driver)
```
The test talks to pages, not to locators.

```
received = form.submit()
```
submit() moves to another page, so it returns that page object.

```
assert received.message() == "Received!"
```
The assertion stays in the test.

```
assert len(fields) == 11
```
Eleven labelled controls on this page.

```
with pytest.raises(ValueError):
```
The constructor checks the page and fails loudly.

5.3 Good practices and things to avoid

Need a browser? Ask first whether a lighter test can do the job. Browser tests are the most expensive kind.

Short and focused Set up the data, perform a few actions, evaluate the result. Many small independent tests beat one grand journey.

Clean start A new driver starts with a new profile. Use a fresh browser per test and never share test data.

Prepare state outside the UI Log in and create data through APIs, then set cookies, instead of clicking through forms.

Avoid CAPTCHAs, download progress, status codes, third party logins, test order, performance numbers, spidering and 2FA are jobs for other tools or test hooks.

```
first.add_cookie({"name": "cart", "value": "3 items"})
```
State a first test leaves behind: a cookie

```
first.execute_script("localStorage.setItem('theme', 'dark')")
```
and a value in local storage.

```
second = webdriver.Chrome()
```
A new driver gets a new, empty profile by default.

```
session = "abc123"
```
In a real suite this value comes from a login API call, not from the login form.

```
driver.get("https://www.selenium.dev/selenium/web/cookies.html")
```
Now open the page under test. It reads its cookies on load.

5.4 Troubleshooting: errors, stale elements and logs

Error names Each W3C error has a class per language. Read the name, then the message, then your line in the trace.

Stale element The node behind a reference was replaced, for example by a refresh. Find the element again.

Context An element found in one frame or window is unknown in another. Switch back before using it.

Client log Python: logger selenium at DEBUG.

Driver log ChromeDriver writes COMMAND and RESPONSE lines when its service is given a log file and a verbose level.

```
email = driver.find_element(By.ID, "email")
```
Found inside the frame.

```
driver.switch_to.frame("iframe1")
```
The fix: switch back to the context where it was found.

```
class Requests(logging.Handler):
```
A tiny handler that prints one line per request.

```
m = re.match(r"(GET|POST|DELETE) http://[^/]+(\S*)", record.getMessage())
```
Request lines start with the method and the URL.

```
log = logging.getLogger("selenium")
```
The parent of every Selenium logger in Python.

```
log.setLevel(logging.DEBUG)
```
Python logs at WARNING by default. Requests are written at DEBUG.

```
service = Service(log_output="chromedriver.log", service_args=["--log-level=DEBUG"])
```
Where ChromeDriver writes its log, and how much.

TRAP The documentation shows the JavaScript logger as getLogger("webdriver"). In selenium-webdriver 4.51 that name printed nothing in our run, while driver.http and driver.http.Executor printed every request. In pytest, the documentation adds that logs of passing tests are hidden unless you run pytest -s -p no:logging.

STEP 6 Scale out with Grid

6.1 Selenium Grid 4

Why a Grid Parallel runs on many browsers, versions and systems, and shorter suites: tests × average time ÷ nodes.

Components Router (4444), New Session Queue, Distributor, Session Map, Node (5555) and Event Bus (4442, 4443).

Roles Standalone for one machine, Hub and Node for most Grids, Distributed for very large ones.

Remote driver Only the driver creation changes. se:name adds metadata, se:cdp gives DevTools through the Grid, and a local file detector sends upload files to the node.

Queue and quit New sessions wait in the queue for a free slot. Always quit, or the slot stays busy until the session timeout.

```
with urllib.request.urlopen("http://localhost:4444/status") as res:
```
The same endpoint a driver has, now answered by the Grid.

```
driver = webdriver.Remote("http://localhost:4444", options=webdriver.ChromeOptions())
```
Returns when the Grid has given this test a slot.

```
time.sleep(3)
```
Pretend the test takes 3 seconds.

```
with ThreadPoolExecutor(max_workers=3) as pool:
```
Three threads, so three tests start at the same moment.

```
options.set_capability("se:name", "upload on the grid")
```
se: capabilities are Grid metadata. se:name shows in the Grid UI.

```
driver = webdriver.Remote("http://localhost:4444", options=options)
```
The only change from a local test: a URL instead of a local driver.

```
driver.file_detector = LocalFileDetector()
```
Sends the local file to the node, which may be another machine.

Suite time on a Grid

$$\text{total time} = \frac{\text{number of tests} \times \text{average test time}}{\text{number of nodes}}$$

6.2 WebDriver BiDi, CDP and Grid features

BiDi A W3C protocol with a WebSocket, so the browser can push events. Turn it on with webSocketUrl, then subscribe to console and page errors.

Events, not polling Handlers are called when events arrive. Wait for them, because they come after the action that caused them.

CDP Chrome and Edge only, no stability promise, the three newest Chrome versions. Use it for Chromium features with no other API.

GraphQL POST a query to /graphql on the Grid for nodes, slots, sessions and the queue size.

Grid endpoints /status, DELETE /session, and drain or remove a node through the Distributor endpoints.

```
options.enable_bidi = True
```

Ask for a BiDi WebSocket next to the normal HTTP connection.

```
driver.script.add_console_message_handler(lambda e: heard.append(f"console {e.level}: {e.text}"))
```

Called for every console.log, console.error and so on.

```
driver.script.add_javascript_error_handler(lambda e: heard.append(f"page error: {e.text}"))
```

Called for every uncaught error in the page.

```
WebDriverWait(driver, 5).until(lambda _: len(heard) == 3)
```

Events are pushed, not returned, so wait until all three arrived.

```
version = driver.execute_cdp_cmd("Browser.getVersion", {})
```

Any CDP method by name, with a dict of parameters.

```
driver.execute_cdp_cmd("Emulation.setTimezoneOverride", {"timezoneId": "Asia/Kolkata"})
```

Something WebDriver Classic has no command for.

```
q = "{ grid { nodeCount, totalSlots, maxSession, sessionCount, sessionQueueSize } }"
```

Ask only for these fields.

TRAP BiDi is still growing. The documentation lists log, network, script and browsing context features, and some network APIs are marked as not yet stable in some bindings, for example the Python docs example uses an underscore method for intercepts. Check the documentation page for your binding before you build on a feature.

STEP 7 Revise and practise

7.1 Revision: predict, debug and design

Predict, then run The driver does what the protocol says, not what the code seems to say. Offsets add, Shift changes case, a refresh makes references stale.

Cause before fix Name why a test is slow or flaky before changing it. Sleeps, mixed waits and missing quit calls are the usual causes.

Layers Tests, page objects and components, a driver factory and support code. Each change should touch one layer.

Size with the formula Suite time = tests × average time ÷ nodes, with about one CPU and 1 GB of RAM per browser.

Know your binding Time is a Duration in Java, seconds in Python and milliseconds in JavaScript, and a missing cookie gives null, None or an error.

Explicit wait poll 500 ms

Implicit wait 0

TRAP A good bug answer has three parts: what the code really does, why that breaks, and a fix that removes the cause. A retry that hides the failure is not a fix.